Check out my portfolio here: https://jack-morgan22.github.io/

Have You Seen This Man?

JACK MORGAN - LEAD GAMEPLAY PROGRAMMER + LEAD QA TESTER

SHOWCASE

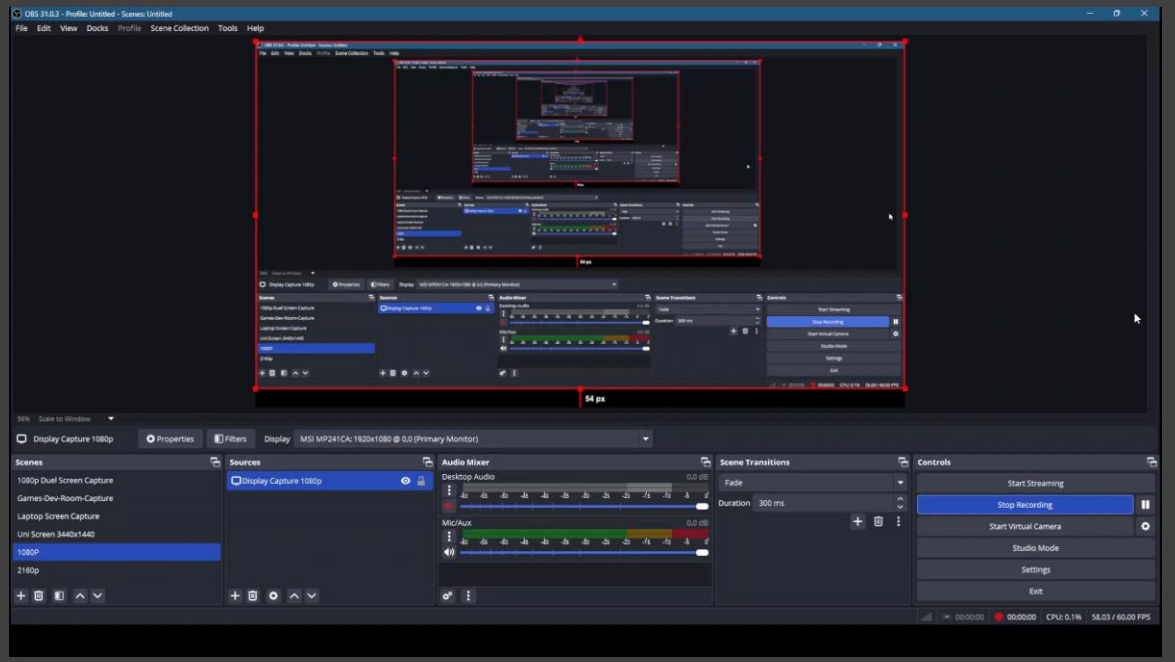
Game Trailer

New Commit (08/05/25)
Fixed Object Interaction, Movement, and Camera Rotation Bug

#1 - Item can collide with player & cause the player to fly away, or cause weird movement behaviour (FIXED)

#2 - Script works in test scene but won't work in actual gameplay scenes (objects will rotate but so will the player's camera) (FIXED)

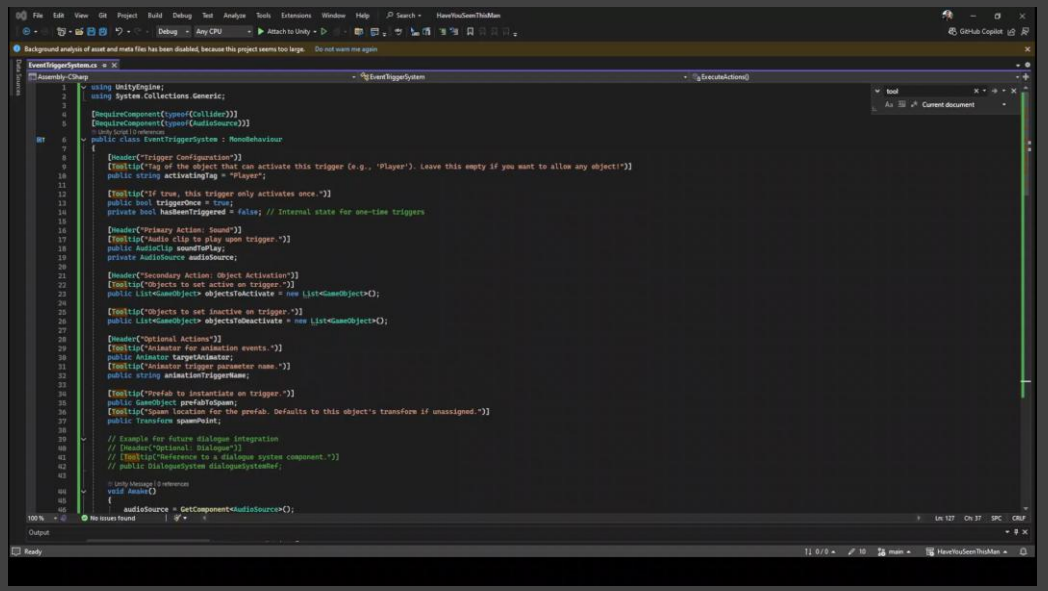
Prefab was incorrectly set up in the STREET_TEST scene. I have set it up again from scratch. Please check that it is set up correctly in case I missed something



New Commit (15/05/25)
Event Trigger System Initial Implementation

EventTriggerSystem.cs script created:

- Core functionality: Plays a sound when the player (or specified tag) enters the trigger collider.
- Designers can attach this script to any Game Object with a Collider (set to 'Is Trigger') and an Audio Source.
- Object Activation/Deactivation: Added the ability to activate and/or deactivate specified Game Objects upon trigger enter.



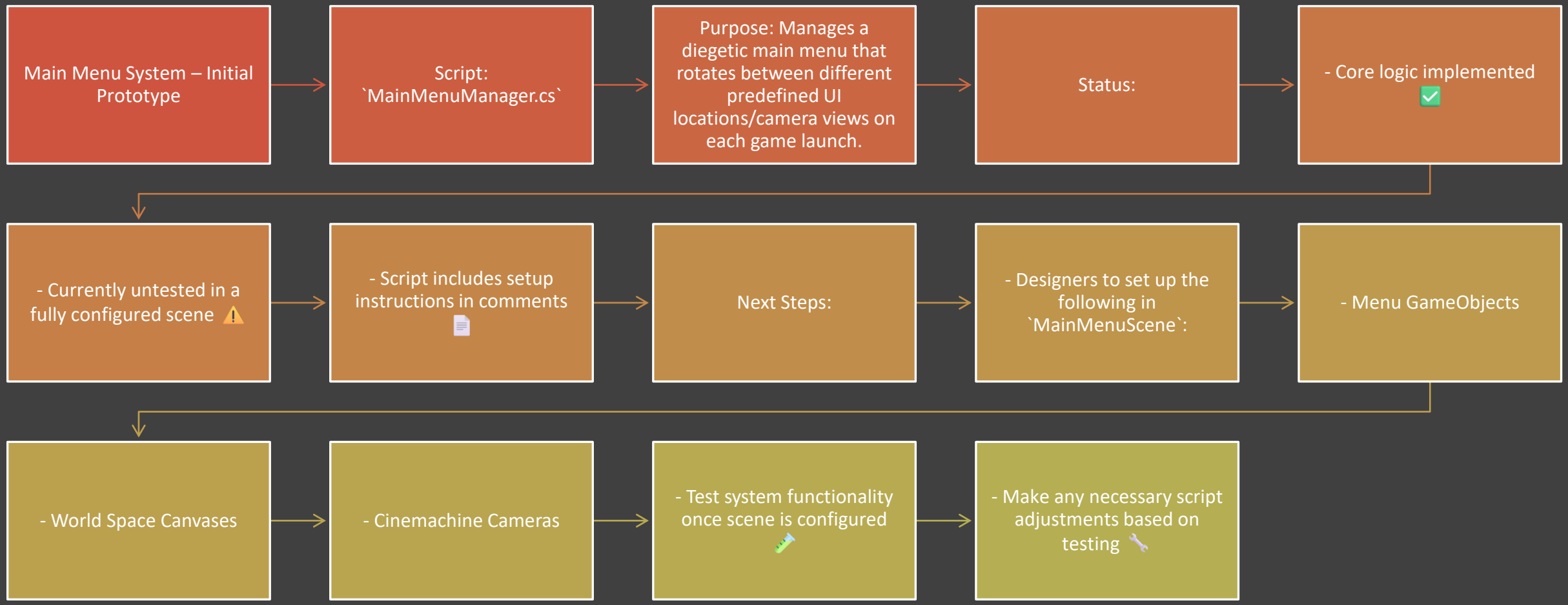
Implemented (Awaiting Full Designer Testing):

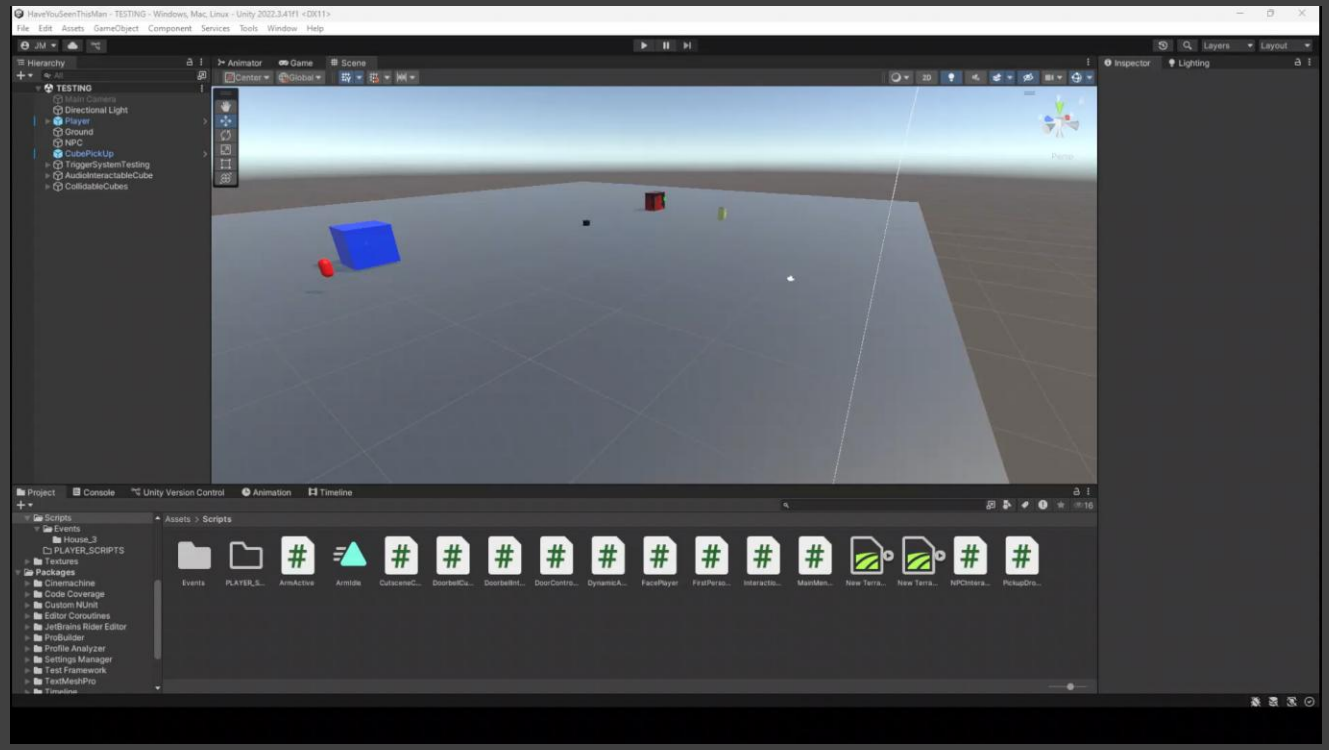
- Animation Triggering: Option to trigger a specified animation on an Animator.
- Prefab Spawning: Option to spawn a chosen prefab at a designated (or default) location.
- (Dialogue triggering is stubbed/commented out for now, pending integration with a dialogue system).

Key Features for Designers:

- Configurable Activating Tag (e.g., "Player", or leave empty for any object).
- Trigger Once option to prevent re-triggering.
- Inspector fields for Audio Clip, lists of Objects To Activate/Deactivate, Animator + Animation Trigger Name, and Prefab To Spawn + Spawn Point.

New Commit (21/05/25)
Added initial prototype for diegetic rotating menu system





New Commit (28/05/2025)
Cleaned up and organised the Testing Scene

NPC Interaction System (28/05/2025)
Initial Implémentation

Modular System for Player-NPC Interactions: Player Control Suspension & Cinematic Camera Transitions

Core Functionality:
- Player stops movement/look on interaction.
- Camera smoothly transitions to/from NPC.
- Player regains control post-interaction.

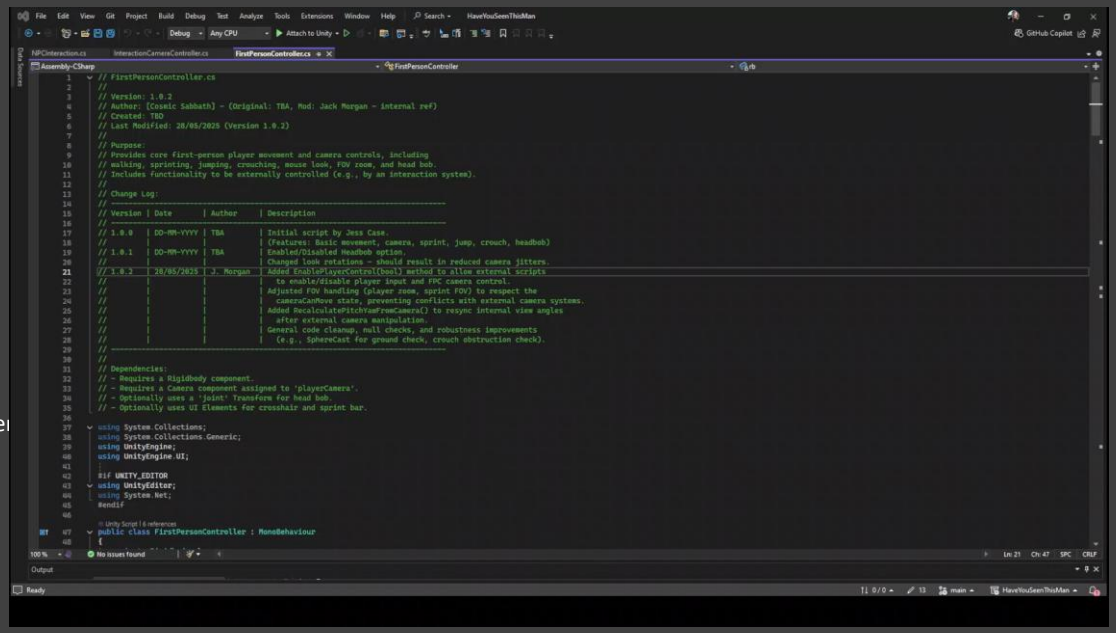
Scripts Created/Modified:
- `FirstPersonController.cs` (Modified - v1.0.2):
  - Updated for external control (input enable/disable, view angle resync) & state handling during interactions.
  - Improved robustness and consistency.

- `InteractionCameraController.cs` (New - v1.0.0):
  - Manages cinematic camera for NPC interactions (zoom, pan, FOV).
  - Restores original player camera state.
  - Framing and transitions are fully configurable.

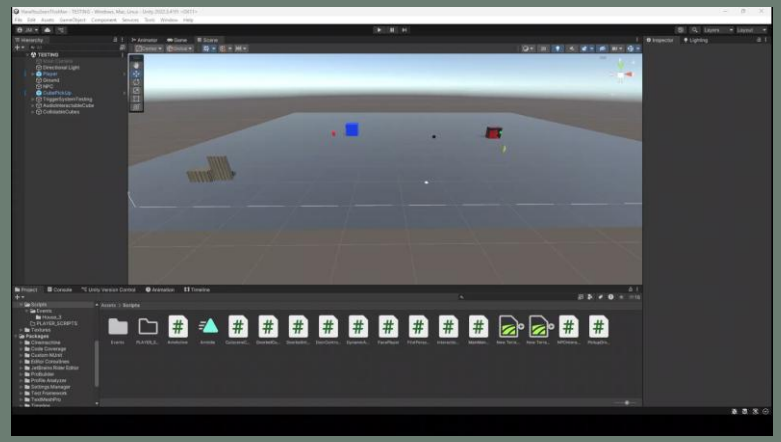
- `NPCInteraction.cs` (New - v1.0.0):
  - Coordinates NPC interaction: player detection, control suspension (via FPC), and cinematic camera transitions (via InteractionCame
  - Placeholder for dialogue integration.
  - Supports custom camera focus points and UI prompts.

Key Features for Designers:
- NPCs are made interactive using `NPCInteraction.cs` + trigger collider.
- Per-NPC configuration for interaction key, pause behaviour, focus points, and prompts.
- `InteractionCameraController` provides designer-friendly cinematic camera control.

Implemented (Awaiting Full Designer Testing & Dialogue Integration):
- Functional control flow: stop > zoom-in > pause > zoom-out > resume.



NPC Interaction System (29/05/2025) Refinement & Code Polish



Refined NPC interaction system:

- Introduced `playerStandoffDistance` to control player proximity to NPC, preventing overly close perspectives.

- Camera now frames via `cameraLocalOffsetDuringInteraction` relative to the player's new standoff position.

- Emphasises subtle FOV change & camera dolly to minimise background stretch (DoF recommended for full effect).

- Verified reliable player control restoration post-interaction.

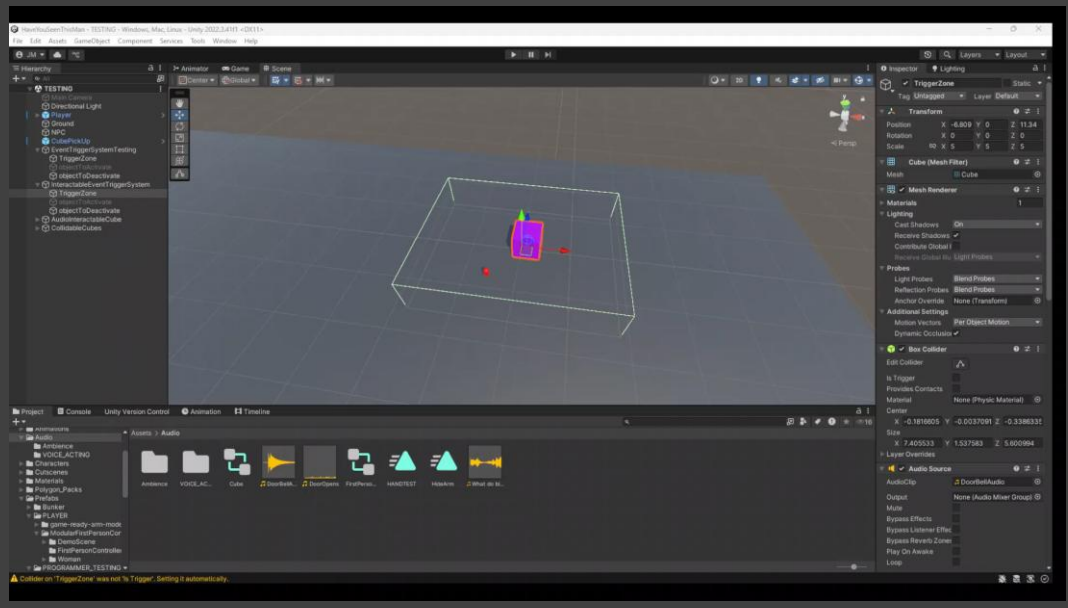
- Improved `InteractionCameraController.cs` code readability by removing old placeholders.

New Commit (05/06/2025)
Implement 'E' Key Interactable Event Trigger System

Introduced a new component: `InteractableEventTriggerSystem`
Designed for player-initiated event activation.

Unlike the existing `EventTriggerSystem` (which activates automatically on entry), this system requires the object with the specified `activatingTag` (e.g., "Player") to press 'E' **while inside the trigger collider**. This allows for more controlled and intentional interaction with game elements.

<u>Key Features:</u>
- Activates events on 'E' key press when the target object is in range.
- Supports all standard event types:
  - Playing Audio Clips
  - Activating/Deactivating Game Objects
  - Setting Animator triggers
  - Instantiating prefabs
- Configurable as one-time or repeatable.
- Automatically ensures the attached Collider is set to "Is Trigger".
- Ideal for interactive objects such as:
  - Doors
  - Pickups
  - Dialogue initiators
  - Levers



New Commit (05/06/2025)
Implemented and created a comprehensive Pause Menu

Features:

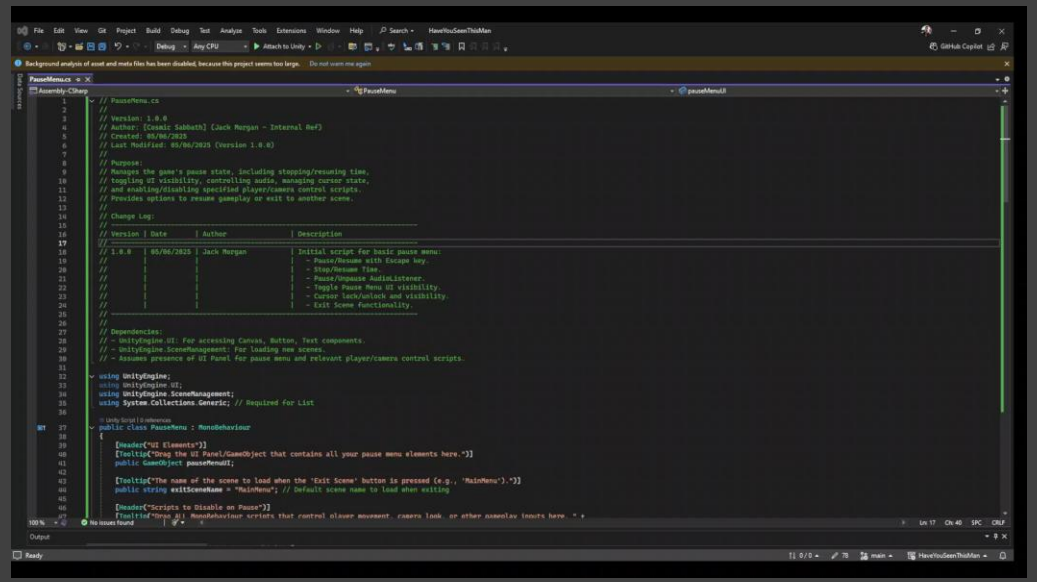
• Intuitive UI:
- Dedicated buttons for Resume Game and Exit Scene.

• Core Pausing Functionality:
- Fully pauses/resumes game time, audio playback, and background gameplay logic.

• Dynamic Control Management:
- Automatically enables/disables specified player and camera control scripts during pause/resume.
- Prevents accidental player input while paused.

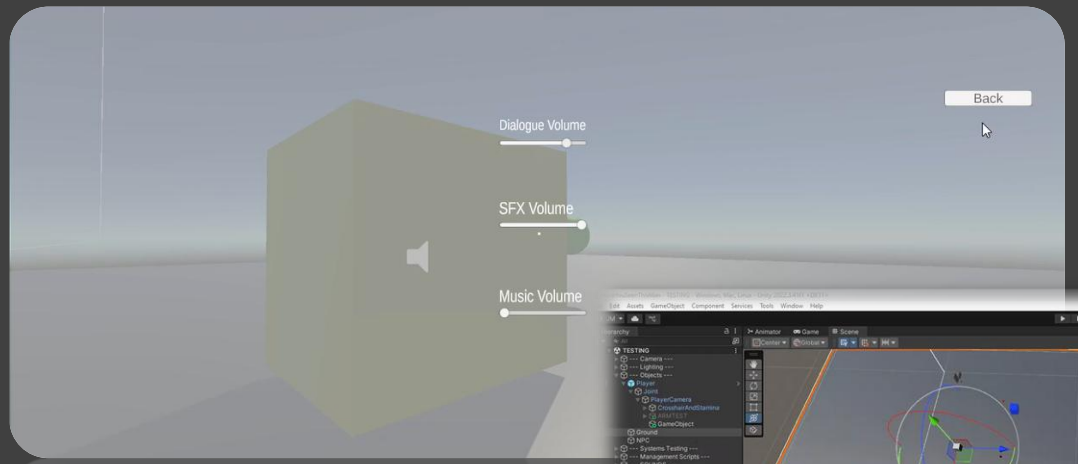
• Cursor & Interaction Handling:
- Manages cursor visibility and lock state for clean menu interaction.
- Unlocks and shows cursor on pause, re-locks on resume.

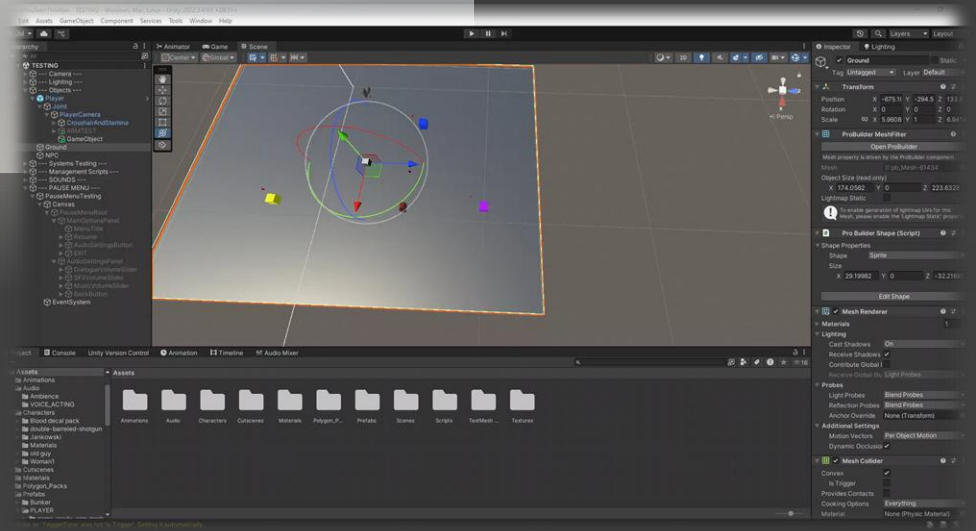
• Seamless Scene Transitions:
- Includes option to exit directly to a designated Main Menu scene.
- Streamlines navigation back to the main gameplay or menu hub.



New Commit (11/06/2025)
Added comprehensive audio volume controls to the Pause Menu

This update introduces a new 'Audio Settings' panel within the Pause Menu, allowing players to independently adjust volume levels for Dialogue, Sound Effects (SFX), and Music.





Key Changes:

• UI Restructure:
- Implemented a nested panel system in the Pause Menu:
`PauseMenuRoot > MainOptionsPanel`, `AudioSettingsPanel`

• Panel Navigation:
- 'Audio Settings' button added to `MainOptionsPanel` to switch views.
- 'Back' button added in `AudioSettingsPanel` to return to main options.

• Audio Mixer Integration:
- Uses Unity's Audio Mixer with exposed parameters:
`DialogueVolume`, `SFXVolume`, `MusicVolume`

• Real-time Volume Sliders:
- Integrated three sliders: `DialogueVolumeSlider`, `SFXVolumeSlider`, `MusicVolumeSlider`
- Sliders adjust mixer volume in real-time, converting linear values to logarithmic dB values.

• Initialisation:
- Sliders auto-initialise to reflect current mixer settings on scene load/menu open.

• Script Updates:
- Modified `PauseMenu.cs` to handle panel switching and link sliders to Audio Mixer controls.

This feature gives players fine-tuned control over their audio experience for improved immersion and accessibility.

New Commit (18/06/2025) Implement a persistent scene loading system

This commit introduces a new, persistent SceneLoader system designed to manage asynchronous scene transitions. This system features a polished loading screen with fade effects and a progress bar, providing a smooth user experience.

Key Changes:

- A new SceneLoader.cs script has been added as a persistent singleton that handles all scene loading. It manages the UI fade-in, asynchronous loading, progress bar updates, and the final fade-out to reveal the new scene.
- The MainMenuManager has been integrated with the new system. I've modified its StartGame() method to use the SceneLoader.Instance for a clean separation of concerns, meaning scene transition logic is now completely decoupled from the main menu UI.
- For proper scaffolding, a SceneLoader Persistent GameObject has been added to the MainMenu scene. This object hosts the SceneLoader script and all associated UI elements for loading.
- Finally, I updated the script headers in both MainMenuManager.cs and SceneLoader.cs with step-by-step setup instructions. This provides clear guidance for future use and integration.

New Commit (01/07/2025)
Implemented Exit Game functionality

This commit adds the essential 'Exit Game' functionality to the main menu, completing a core requirement of the initial task.

Key Changes:

• New `QuitGame()` Method:
  - Added to `MainMenuManager.cs` as a public method.
  - Designed to be triggered by any 'Exit' or 'Quit' button within the UI.

• Editor & Build Compatibility:
  - Uses preprocessor directives (`#if UNITY_EDITOR`) to ensure correct behaviour:
    • Stops Play Mode in the Unity Editor.
    • Calls `Application.Quit()` in built versions.
  - Ensures smooth testing and user experience across environments.

• Updated Documentation:
  - Script header and change log in `MainMenuManager.cs` updated to version 1.2.0.
  - Reflects the addition of this feature and outlines setup steps.

New Commit (02/07/2025)
Add Options Menu Framework

This commit introduces the necessary framework to support a dedicated Options Menu panel within the Main Menu. The new logical flow allows for seamless toggling between the active direction menu and the options UI without requiring a scene change, providing a foundation for all future settings implementation.

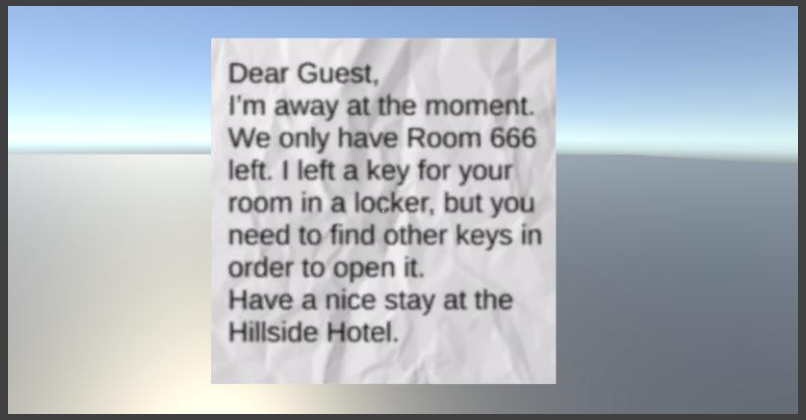
Key Changes & Enhancements:

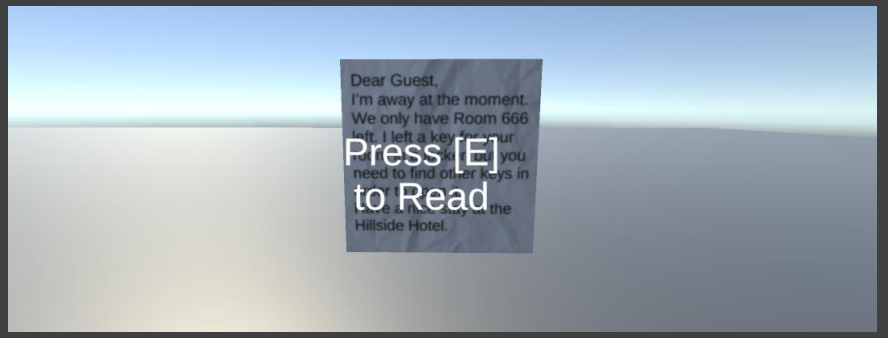
- New ToggleOptionsMenu() Method: A new public method has been added that handles opening the options panel while hiding the main menu, and then restoring the main menu upon closing.
- Active Menu Tracking: The activeMenuObj and activeInitialCamera are now tracked after the initial random selection. This ensures that the correct objects hide and show when toggling options.
- New Inspector Slot: We've added a serialized field for an OptionsViewCanvas GameObject, allowing designers to easily link the UI panel from the scene.
- Comprehensive Documentation: The script header, changelog, and setup instructions have been updated to version 1.3.0. This documentation guides designers in setting up and using the new feature.

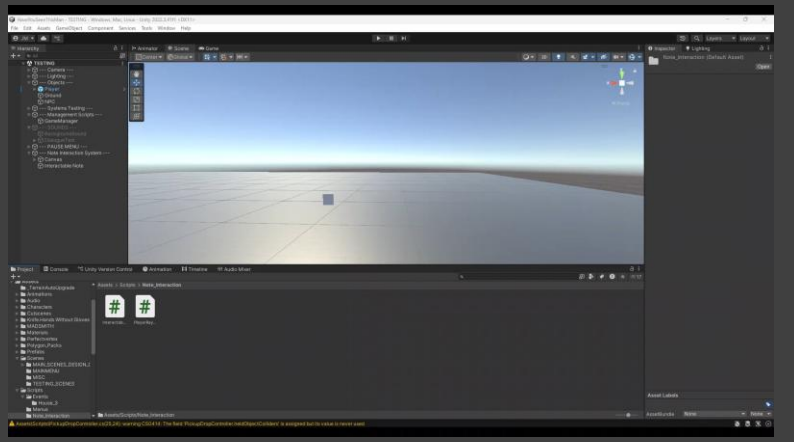
This architectural enhancement creates a robust and user-friendly way to handle in-game settings directly from the main menu.

New Commit (10/07/2025) – Implement Raycast Note Interaction System

This commit introduces a complete, raycast-based system that allows the player to interact with and read notes found in the game world. This provides a flexible and efficient foundation for delivering lore, clues, and other narrative content in an immersive manner.







Key changes include:

New Script: PlayerRaycastInteraction.cs: The core controller, placed on the player. It handles all raycasting logic, input detection, displaying the UI prompt, and coordinating with the FirstPersonController to disable/enable player controls.

New Script: InteractableNote.cs: A simple data component to be placed on any note object. It holds the note's Material and an optional AudioClip, making it easy for designers to define the content of each note.

Streamlined Designer Workflow: The system uses a single Material asset for both the note's in-world appearance and its full-screen UI display. This simplifies the content creation pipeline significantly.

Dynamic Note UI: Upon interaction, the system dynamically creates a full-screen UI to display the note's texture, providing a focused "reading" state. Pressing the key again closes the view and restores control.

Comprehensive Documentation: Both new scripts have been fully documented with header comments, changelogs, and embedded "How to Use" guides to assist designers in implementing new notes.

This feature provides a robust and modular way for players to engage with narrative elements directly within the game environment.

Status & Next Steps: The C# framework for the note interaction system is complete and functional. The next steps are for designers to create the InteractionPrompt UI element and link it to the Player prefab, then begin populating levels with interactable notes following the new documentation.

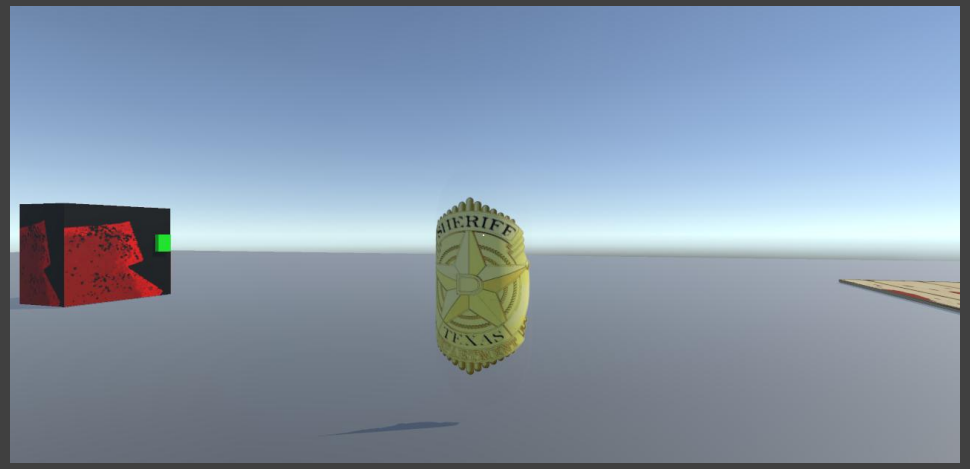
New Commit (11/07/2025) Expand Cutscene Trigger with Permanent Object Spawning and Despawning

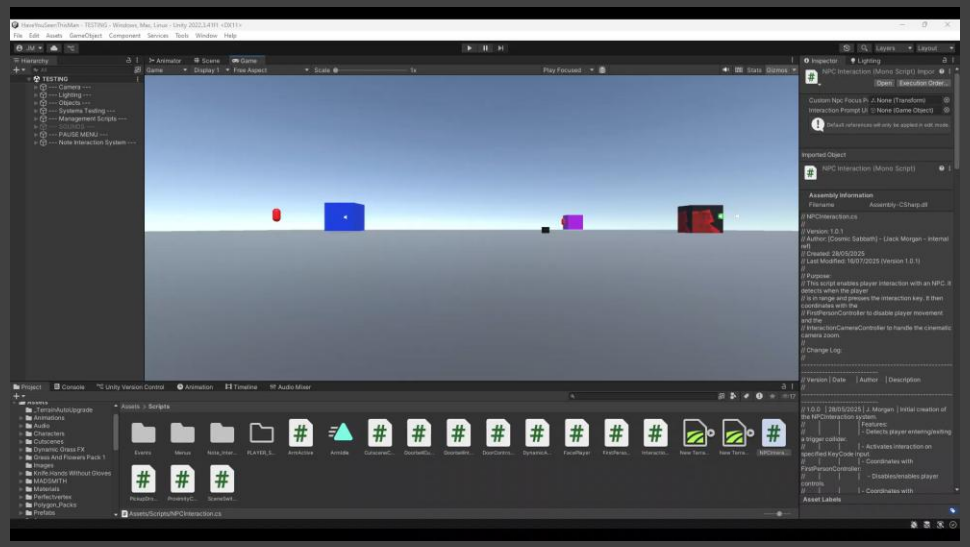
The existing CutsceneTest script has been significantly upgraded. It's no longer a simple toggle but a versatile in-game event trigger that enables permanent object spawning and despawning. This change allows designers to create dynamic world events without writing any code, such as revealing a key item or permanently clearing a blocked path.

The script now includes new features for permanent spawning and despawning. A new ObjectsToSpawn array lets designers instantiate prefabs, while the optional Spawn Points array allows for precise placement. Similarly, the ObjectsToDespawn array can permanently destroy Game Objects.

I’ve also updated the event logic so that permanent actions now process before temporary toggles, streamlining the workflow. Designers can configure both permanent and temporary effects within a single component, requiring no code for complex interactions.

This update includes comprehensive documentation with a new header, changelog, and an embedded "How to Use" guide for designers. The result is a robust, reusable tool that reduces the need for custom one-off scripts and empowers level designers to build complex narrative and gameplay sequences.





New Commit (16/07/2025) - Fixed Critical NPC Interaction State Bug

Addressed a critical bug in the NPCInteraction system where the player would remain permanently "in range" of the first NPC they interacted with. This caused any subsequent interaction key presses, regardless of the player's location, to incorrectly re-engage the interaction with that original NPC. The system now correctly resets its state when the player leaves an NPC's vicinity, ensuring interactions only occur when intended.

Key changes include:

- **Fixed Persistent State Bug in NPCInteraction.cs:** Corrected a core logic error where the script would set _playerInRange to true upon entering a trigger but never reset it. This caused the first NPC interacted with to become the permanent interaction target for the entire scene.
- **Implemented OnTriggerExit Logic:** Added the OnTriggerExit Unity message to the NPCInteraction script. This ensures that when the player leaves an NPC's trigger collider, the _playerInRange flag is properly set to false and the cached player controller reference is cleared.
- **Improved System Robustness:** The interaction prompt UI now reliably disappears when the player leaves the trigger area, and the system is prevented from starting a new interaction with an out-of-range NPC.
- **Updated Documentation:** The script's internal changelog (Version 1.0.1) has been updated to reflect this critical fix, ensuring team members are aware of the change and its resolution.
- This fix resolves a major gameplay-disrupting issue and ensures the NPC interaction system is now robust and reliable. Interactions are now correctly scoped to the player's immediate proximity, behaving as designers originally intended

New Commit (20/07/2025) - Implemented Core Pistol Controller System

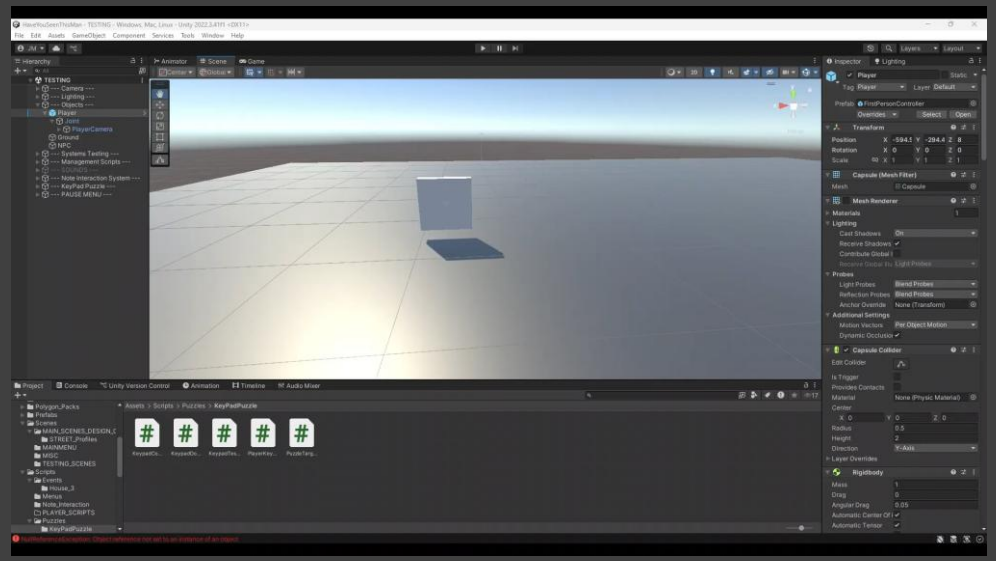
Introduced the PistolController.cs script, establishing the foundational system for player-held firearms. This initial implementation provides all the essential mechanics for a basic pistol, including firing logic, effects management, and a limited ammo clip.

Key changes include:

- **New Script PistolController.cs:** Created and implemented a new, self-contained script to manage all pistol-related actions. The script is designed to be attached directly to the weapon's GameObject.
- **Input-Driven Firing:** The script listens for the standard Fire1 input (Left Mouse Button) to trigger the firing sequence.
- **Integrated Effects System:** The controller plays a designated ParticleSystem (for muzzle flash) and an AudioClip (for the gunshot sound) upon firing, with public Inspector slots for easy asset assignment.
- **Limited Ammo Clip:** A simple ammo system is included, limiting the player to a set number of shots (default is 6). Firing is automatically disabled when the clip is empty.
- **Automatic Reload on Equip:** The script utilises the OnEnable function to automatically reset the ammo count to its maximum value, simulating a reload whenever the weapon is equipped or activated.
- **Comprehensive Documentation:** The script includes a detailed header comment block outlining its version, purpose, changelog, and dependencies, adhering to established project standards.
- This commit establishes a crucial piece of core gameplay. The PistolController is now a fully functional and robust component, ready for integration into the player's inventory and for designers to customise with different sounds and visual effects. This modular design will streamline the addition of other firearms in the future.

New Commit (23/07/2025) - Complete Keypad Puzzle System Implemented

Introduced a complete, modular Keypad Puzzle system. This feature allows for the creation of locked doors that require players to investigate an area to find a numerical code. The system is fully interactive, featuring a diegetic UI, distinct audio feedback for all actions, and seamless management of player controls (disabling movement/camera and handling cursor state) during interaction. The entire puzzle is self-contained and can be easily configured by designers to set different codes and link to various unlockable objects via UnityEvents.



Key changes include:

- **Creation of KeypadController.cs:** This script serves as the central logic hub for the puzzle, managing the correct code, UI state, input handling from UI buttons, and firing onCorrectCode / onIncorrectCode UnityEvents.
- **Implementation of PlayerKeypadInteraction.cs:** Placed on the player, this script uses a raycast to detect and initiate interaction with the keypad via the 'E' key, calling the keypad's BeginInteraction() method.
- **A Simple KeypadDoorController.cs:** This script acts as a receiver for the puzzle's success event, containing a public method to trigger a door's open animation. This demonstrates the modular design of the system.
- **Full UI and Audio Integration:** A functional on-screen keypad was created with a display and buttons. The system includes unique, non-overlapping sound effects for button presses, success, failure, and clearing input, providing clear feedback to the player.
- **Robust Player State Management:** The system automatically disables the FirstPersonController and manages the cursor's lock state and visibility when the keypad UI is active, ensuring a smooth and bug-free user experience.
- **Flexible Exit Options:** Players can close the keypad interface using either the 'Escape' key or a dedicated 'Exit' button on the UI, accommodating common player expectations.
- This feature provides a robust and reusable puzzle template that can be easily deployed and customised by designers for various scenarios throughout the game.

06/08/2025 – QA Bug Report

I have pointed out as many things as possible that need to be fixed. My two biggest issues were lighting and sound in the basement during the build version. I also think that the jump mechanic needs some work to avoid players getting stuck.

Have You Seen This Man – QA Bug Report – Jack Mogan – 06/08/2025

Have You Seen This Man | *PC version 1.4*

Issue	*Scenario*	*Expected Result*	*Actual Result*	*Visual Example*
The Start Button on the Main Menu opens up three Start Button options	Press the Start button on the Main Menu.	Pressing the Start Button should open the scene loader scene, which leads onto the street.	Three new Start buttons open up in a column.	View Photo: https://drive.google.com/file/d/1YDwX0I3Z7f3o9WGVU73ExWsiuWIhYzyq/view?usp=drive_link
The Load Screen has the Start Button overlapping the screen.	Press the functional Start button on the Main Menu. The loading screen will then slowly load up.	The Start button should disappear upon the loading screen appearing.	The Start Button overlaps the load screen.	View Photo: https://drive.google.com/file/d/1WwMDIUtPhrMTw9jXY81SgmfUHAhcCO71/view?usp=drive_link

New Commit (07/08/2025) - Added Pause Menu to Bunker and Street

- Added Pause Menu to Bunker and Street

- Fully added to Bunker and working.

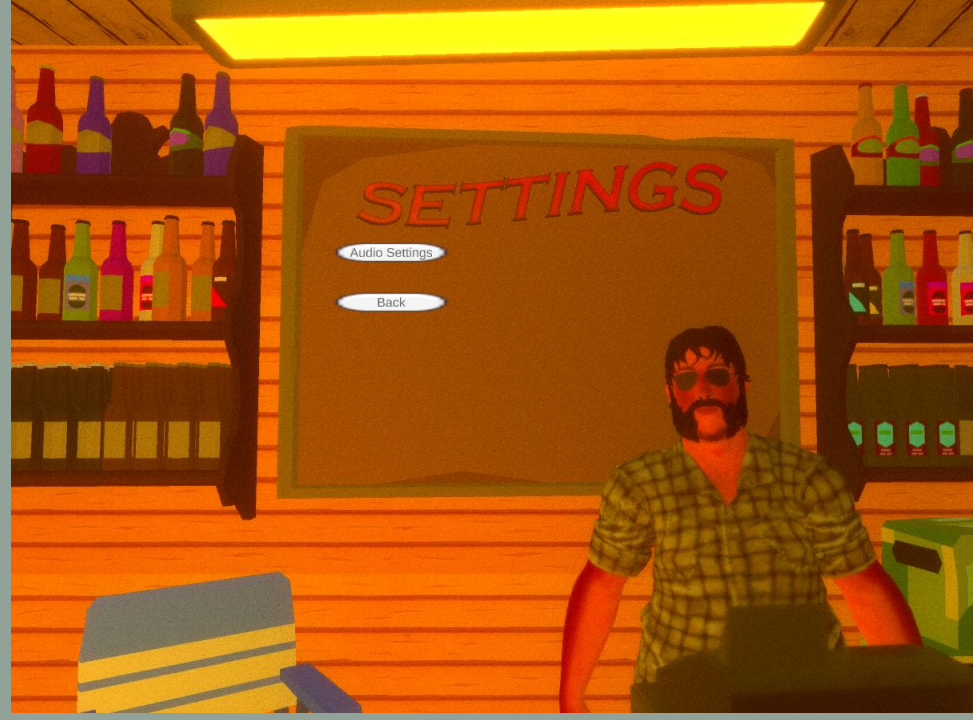
- Fully added to Street. Stops Player Movement and functions correctly, but does not disable camera movement.

New Commit (10/08/2025) Finished the complete Main Menu Settings system

I've finished building the complete settings system for the main menu. I made it so the whole thing runs from one central script (MainMenuSettings.cs) on the GameManager, so it's easy to find. It handles all the panel switching smoothly and even has a clear way to add more settings pages later.

Key changes I made:

- **Created MainMenuSettings.cs:** I made this script to act as the central brain for all the settings logic.
- **Smart Panel Navigation:** I wrote a SwitchToPanel function that takes care of all the panel navigation. It automatically makes sure only one panel is visible at a time, which stops any messy UI from overlapping and will make adding a graphics menu a piece of cake later on.
- **Automatic Background Hiding:** Added a really cool feature: a list in the Inspector where you can just drag and drop stuff (like the main menu buttons) to have them automatically hide when the settings menu is open. Keeps the screen looking clean.
- **Full Audio Settings:** I pulled in the audio settings system I'd already built for the pause menu. All the volume sliders for Dialogue, SFX, and Music are fully working.
- **Reliable Startup:** I squashed a bug where the menu would sometimes appear on load. I'm using a coroutine now to force it to hide after the first frame, so it reliably stays hidden until it's opened.
- **Reliable Startup:** Comments are easy to read and understand, so that you can look at the code and know what is going on



New Commit (10/08/2025) - Refactored Settings across Pause Menu and Main Menu into one MenuManager and made Menu Settings Sync

I've completed a major refactor of the entire settings system to fix inconsistencies and make it more robust for future additions. The old approach had separate, duplicated logic in the PauseMenu and MainMenuSettings, which was causing settings to not sync up. This new system solves that completely.

Key changes I made:

- Introduced a MenuManager: Created a new, persistent script (MenuManager.cs) that is now the source of all game menu settings. It handles all saving/loading via PlayerPrefs and direct communication with the AudioMixer.

- Refactored PauseMenu & MainMenuSettings: Stripped out all the old, duplicate audio logic from these scripts. They are now much simpler UI controllers that just talk to the MenuManager instead of handling settings themselves. This makes them cleaner and easier to maintain.

- Ensured Settings Consistency: This solves the major issue where settings changed in the pause menu wouldn't be reflected in the main menu (and vice versa). Now, any change is updated globally and saved, persisting correctly across scenes and game sessions.

All Menus now fully work and are in sync.

I have also added the Pause Menu to the Crime Scene and Hallway, as well as fixing existing Menus in Street and Bunker, with the exception of the camera movement issue in the Street scene.

New Commit (10/08/2025) - Fixed Generator Turn On

Configured the interactive fuse box in the POWER_EVENT group to have a multi-step repair sequence. This creates the core gameplay loop for restoring power in the level.

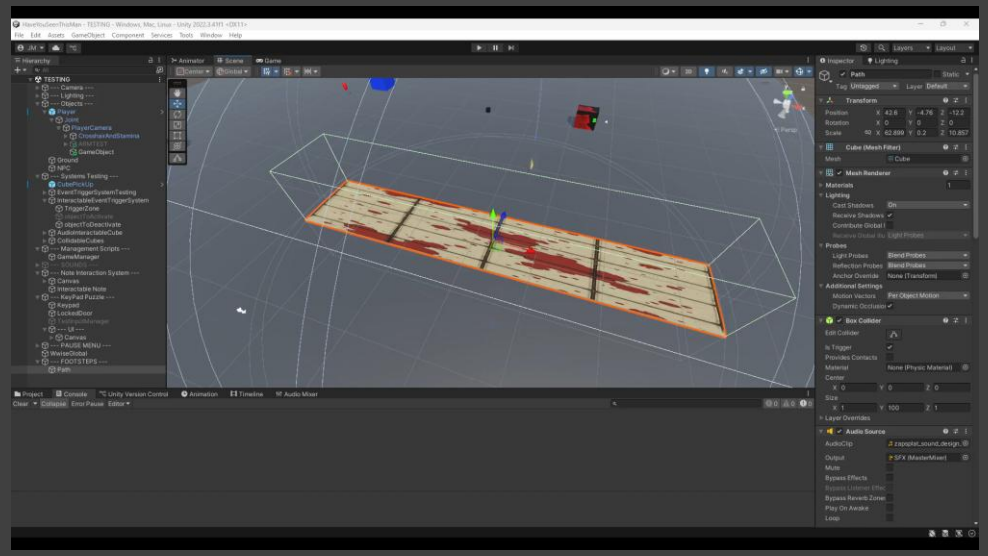
Key changes I made:

- Implemented State Transitions: Used the ToggleObjectsOnInteract script to manage the flow from FuseBox_START -> FuseBox_BROKE -> FuseBox_FIXED.
- Linked Visuals and Effects: Each state now correctly enables/disables the associated GameObjects, including the different fuse box models, red/green lights, and the spark particle effect.
- Completed Interaction Loop: The final FuseBox_FIXED state is correctly non-interactive, ending the sequence once the puzzle is solved.

Known Issue: The interaction sounds (SPARK and SCREWIN) are not playing when their GameObjects are enabled. The cause for this is currently unknown and needs to be investigated.

New Commit (15/08/2025) - Fixed Player Gravity and Stopping Time

Player controls have been updated to make stops feel quicker and more natural. Additionally, I've adjusted the in-game gravity based on conditions such as walking up walls and vertical surfaces to prevent players from being able to scale upwards.

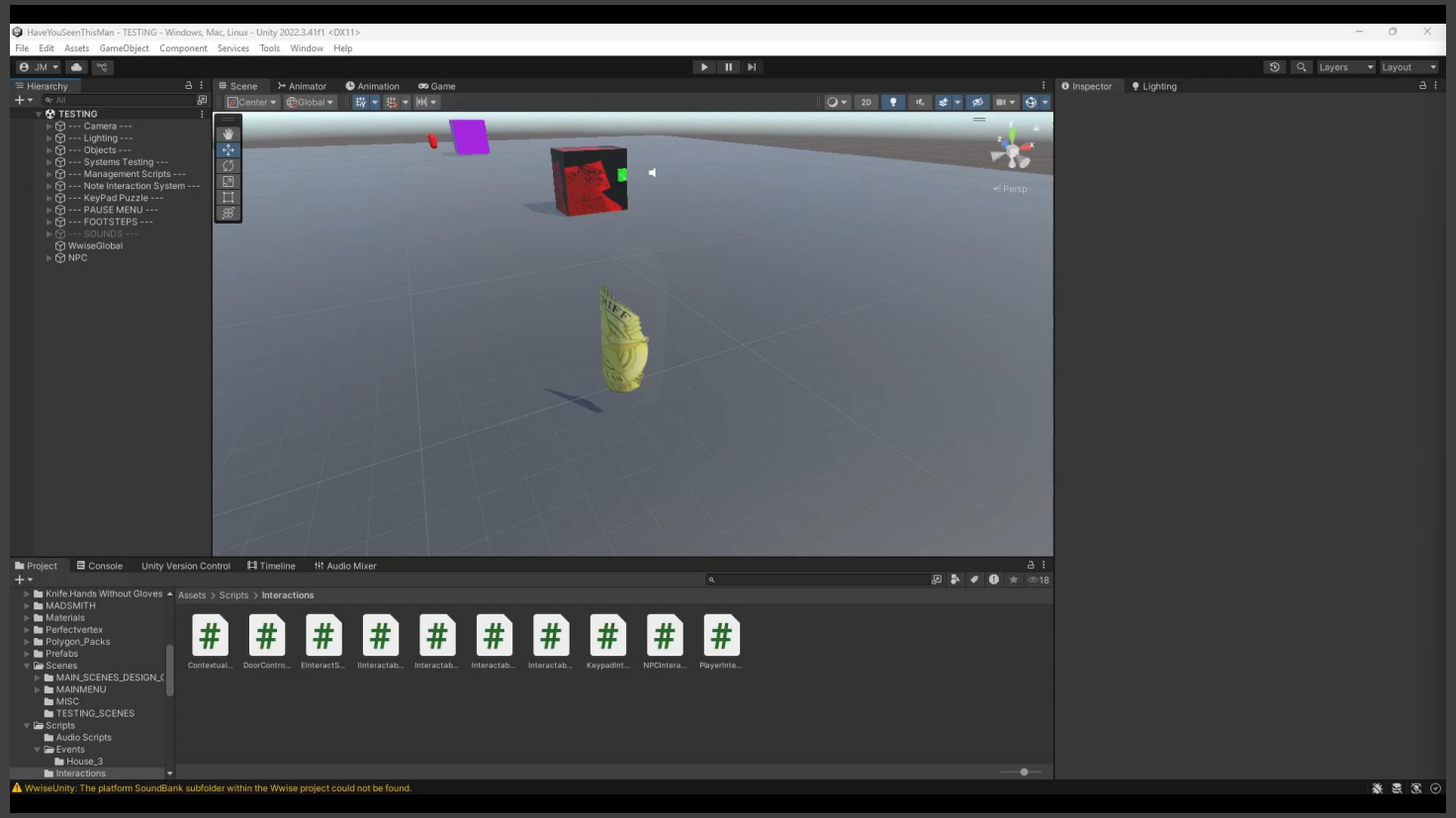


New Commit (26/08/2025) - Implemented Zone-Based Footstep Sound Trigger System

Implemented the FootstepSystem.cs script, which dynamically plays footstep sounds when the player enters a designated trigger zone. This system allows for the creation of distinct audio surfaces (e.g., grass, concrete, mud), thereby enhancing environmental immersion and player feedback.

Key Features:

- **Trigger-Based Activation:** Utilised an OnTriggerEnter/OnTriggerExit system to detect when the player is inside a defined audio zone. The script requires a collider to be set as an Is Trigger.
- **Movement Detection:** Implemented input detection to play sounds only when the player is actively moving (via WASD keys) within the trigger volume.
- **Alternating Audio Clips:** Created a system that alternates between two distinct AudioClip assets with a configurable delay. This prevents repetitive audio and simulates a more natural walking cadence.
- **Instant Audio Stop:** Ensures audio playback stops immediately upon the player exiting the trigger zone, providing a clean and responsive feel.
- **Inspector Configurability:** Exposed all key parameters (footstepSound1, footstepSound2, stepDelay) in the Inspector for easy customisation and reuse of the system for different surface types.



New Commit (01/09/2025) - Refactored Player Interaction System into a Unified Architecture

Implemented a complete architectural overhaul of the player interaction system. The previous fragmented approach has been replaced with a centralised and robust system based on a "separation of concerns" principle. This new architecture improves performance and stability, and a new IInteractable interface makes adding future interactable objects a simple and error-proof process.

The system is now divided into two core components:

- **The "Eyes" (ContextualIconSystem.cs): Handles all visual detection and UI feedback.**
- **The "Hands" (PlayerInteractionController.cs): Handles player input and triggers the interaction.**

Key Architectural Changes & Impact

Universal IInteractable Interface: A new IInteractable.cs interface was created to act as a standard contract for all interactable objects, simplifying communication between the player and the world.

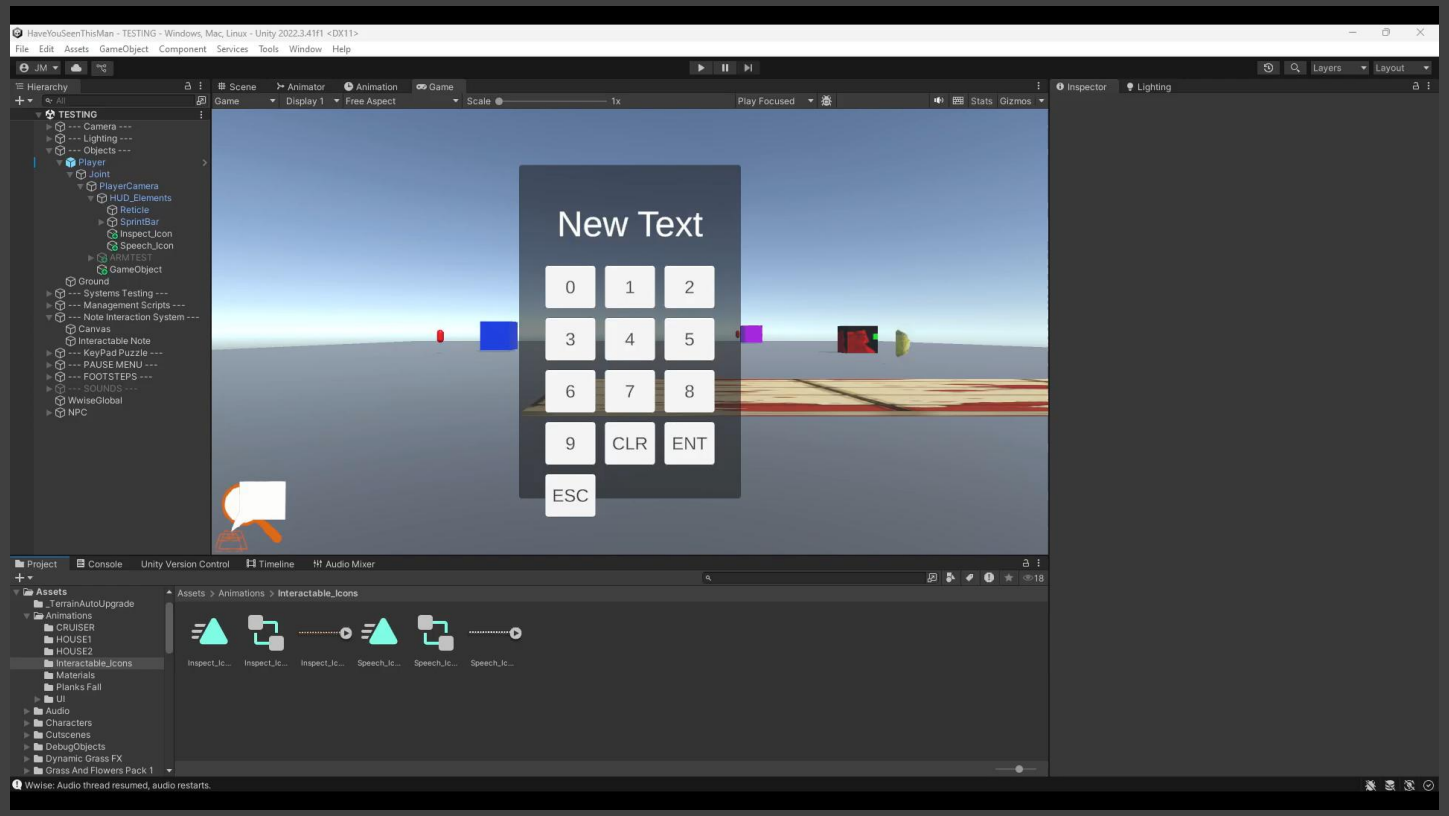
Upgraded Detection Reliability: Upgraded the core detection mechanism from Raycast to a more forgiving SphereCast, making interactions with small or thin objects significantly more reliable and resolving previous "hit and miss" collision issues.

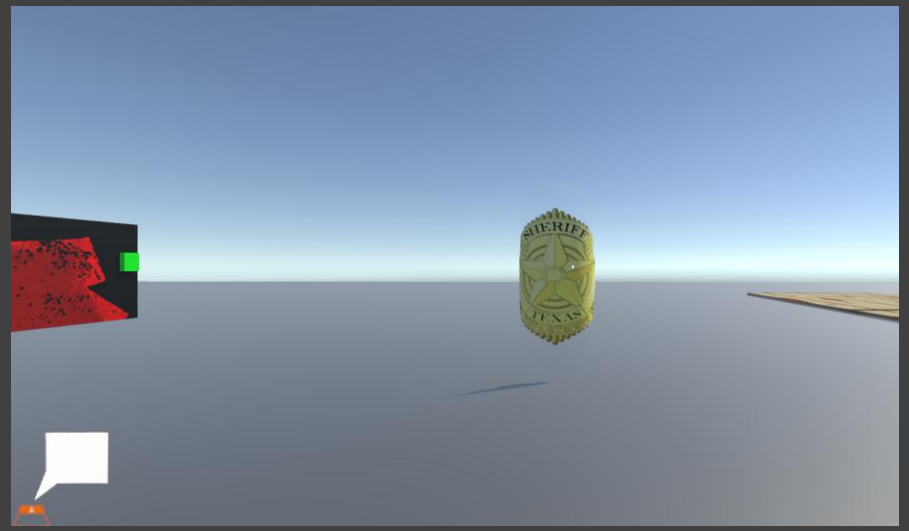
Refactored Interactable Objects: All existing interactable scripts (NPCInteraction, DoorController, InteractableNote, etc.) were refactored to conform to the new system, removing their redundant detection logic and making them more self-contained.

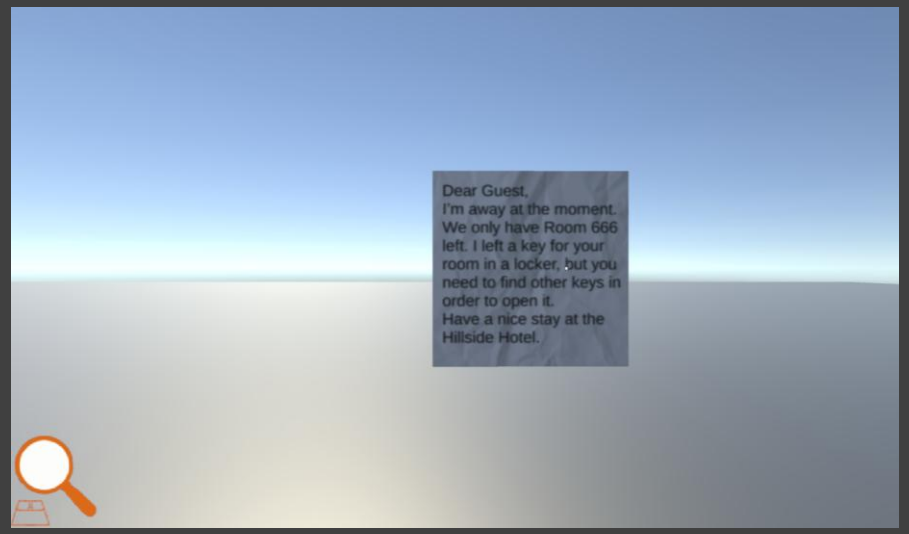
Project Cleanup: Obsolete scripts like IconManager and PlayerKeypadInteraction were removed, and all related scripts were updated with professional documentation.

New Commit (01/09/2025) - Implemented Animated Contextual Icon System

Implemented a new, self-contained system to provide clear visual feedback by displaying a contextual UI icon when the player looks at an interactable object. This feature enhances the user experience by unambiguously indicating points of interest and supports animated icons (via sprite sheets) for a more dynamic and polished feel. This entire system is centralised within the new ContextualIconSystem.cs script.







System Features & Implementation Details

Animated Icon Support: The system simulates GIF-like animations using sprite sheets and dedicated Animator Controllers. Each unique icon can have its own looping animation.

Centralised and Configurable: All available icons are defined in a single list within the ContextualIconSystem script. Designers can easily add new icons and define their unique string IDs in the Inspector.

Data-Driven by Objects: Interactable objects are now responsible for their own visual representation via the InteractableIconProvider.cs component, where designers can specify which Icon ID and Animator Controller to use.

Robust Animation Playback: Solved a common Unity bug by implementing a coroutine that forcefully restarts an icon's animation (Animator.Play()) each time it is displayed, ensuring animations play reliably every time.

New Commit (04/09/2025) - Fixed the following Systems in the Bunker

Key

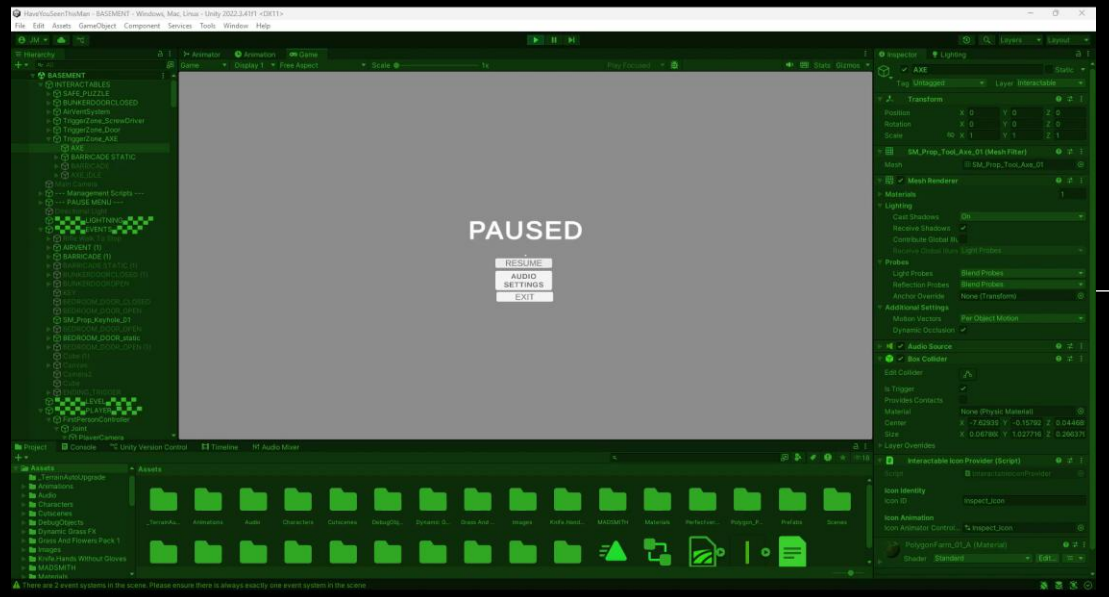
Axe

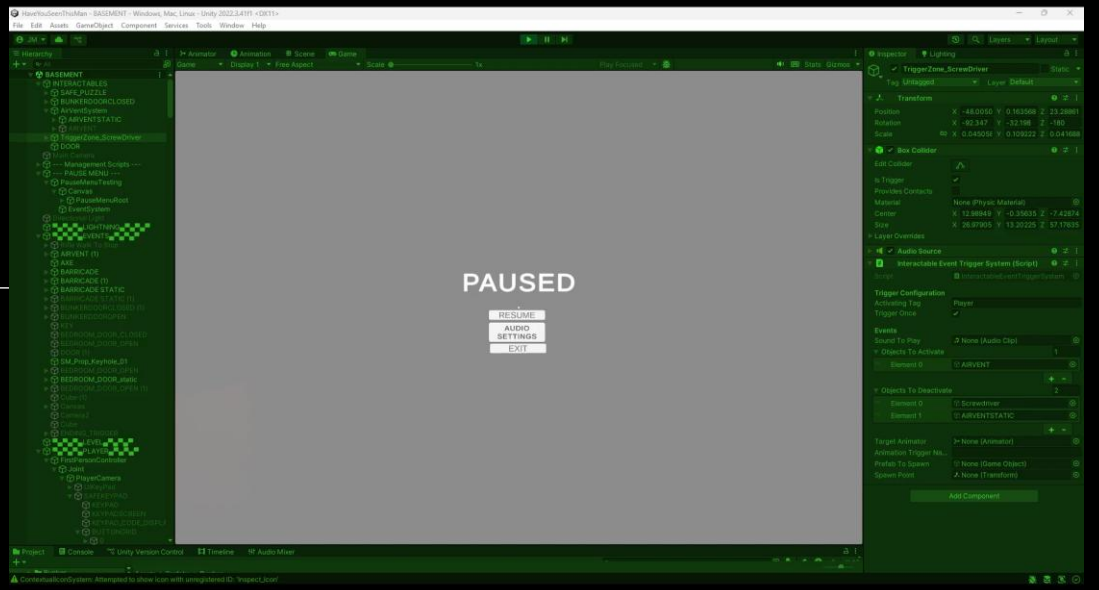
Door

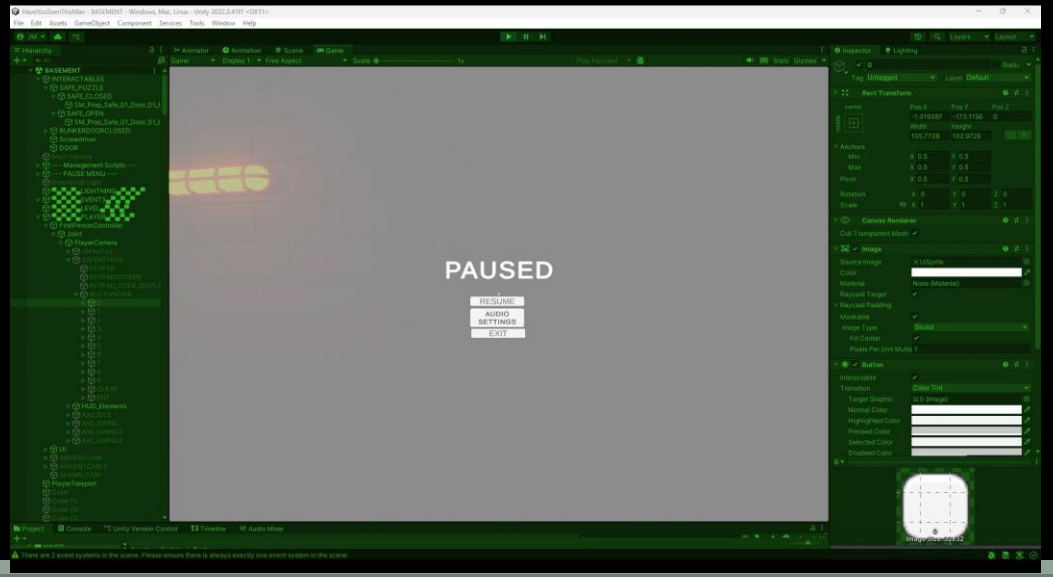
Screwdriver

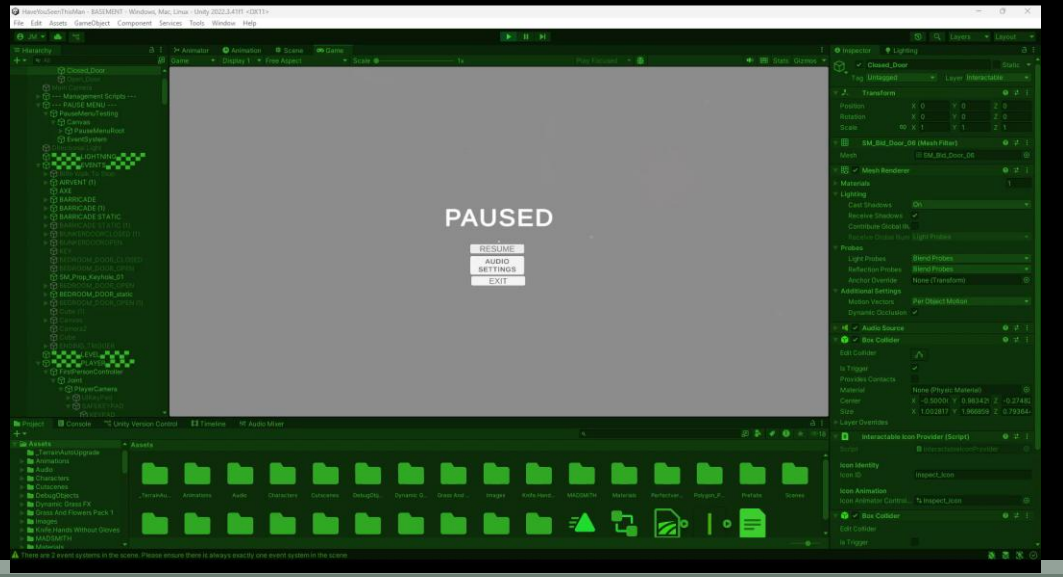
Keypad Safe Puzzle

Implemented the Visual Interaction Indicators.

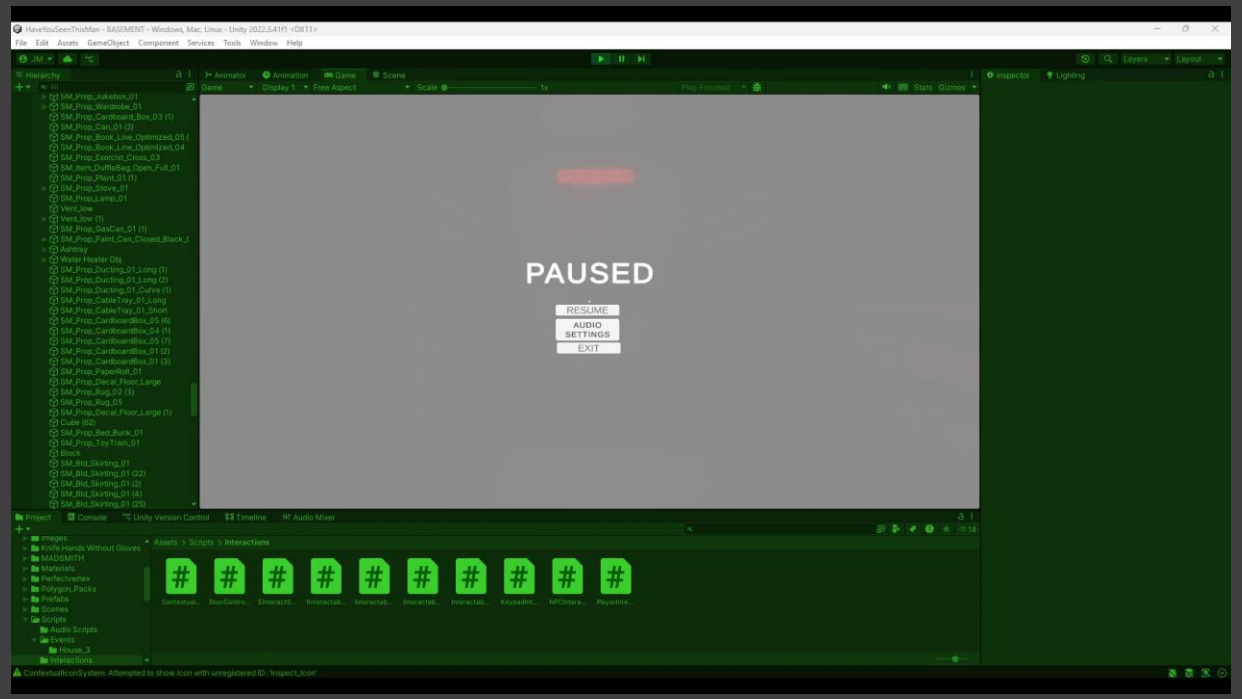

PAUSED
RESUME
AUDIO SETTINGS
EXIT


PAUSED
RESUME
AUDIO SETTINGS
EXIT


PAUSED
RESUME
AUDIO SETTINGS
EXIT


PAUSED
RESUME
AUDIO SETTINGS
EXIT

New Commit (05/09/2025) - Overhauled ToggleObjectsOnInteract.cs into a Multi-Stage State Machine



The existing ToggleObjectsOnInteract.cs script has been completely overhauled, refactoring it from a simple toggler into a robust, multi-stage state machine.

This was necessary to integrate the script into the new unified interaction system and to support complex, sequential puzzles (e.g., 'Start -> Broke -> Fixed') that the original version could not handle. The main benefit is that designers can now set up an entire sequence of events from a single, highly configurable script.

Key Features & Improvements:

- Rebuilt as a State Machine: Designers can now define an ordered sequence of states in the Inspector. Each player's interaction progresses the puzzle to the next stage.
- Centralised Logic: A complex, multi-step puzzle can now be managed by a single instance of this script, greatly simplifying the setup and debugging process.
- Added Flexibility: A new Inspector option allows the sequence to either stop permanently on the final state or loop back to the beginning, making it versatile for both one-off puzzles and simple toggles.
- Full System Compatibility: The script now implements the IInteractable interface, ensuring it works seamlessly with the PlayerInteractionController for activation and the ContextualIconSystem for visual feedback.

New Commit (28/09/2025): Refactor & Fix: Keypad System and Safe Animation

This commit addresses critical interaction and animation bugs within the puzzle systems, stemming from a necessary refactor to improve their robustness. The safe door now animates correctly, and a regression that made the keypad UI unresponsive has been resolved.

Key Changes & Fixes:

Keypad UI Button Fix: Fixed a regression where the keypad's number, clear, and enter buttons were unresponsive. This was caused by their On Click() events becoming disconnected after the KeypadController was moved to a separate GameObject. All UI events have been re-linked to the appropriate methods (AddDigit, ClearInput, CheckCode) on the central KeypadController script, restoring full functionality.

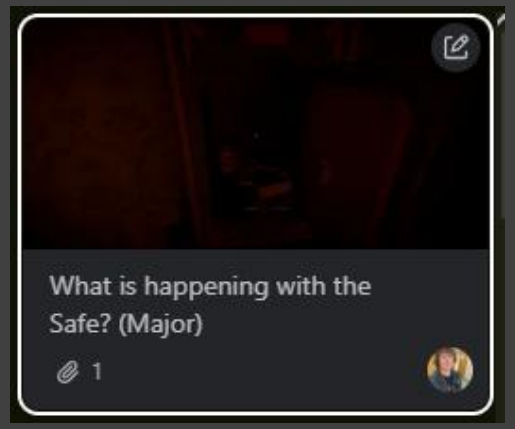
Safe Door Animation Fix: Resolved a bug causing the safe door to open immediately on scene load. The Animator Controller has been corrected to include a default Idle_Closed state, preventing the open animation from playing until it is explicitly triggered by a successful code entry via the onCorrectCode event.

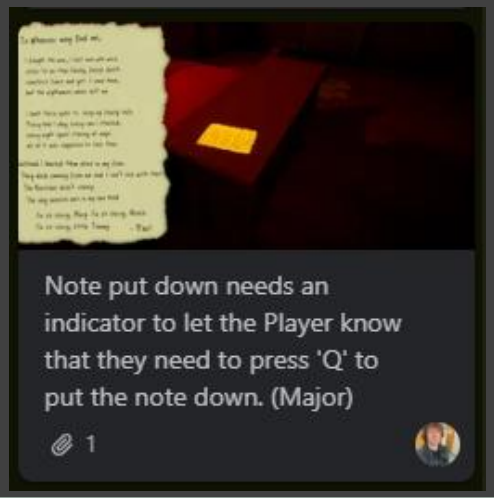
Puzzle System Refactor: The KeypadController script and its associated interaction collider have been moved from the animated safe door to a dedicated, stationary Keypad GameObject. This architectural change decouples the puzzle logic from its visual result, preventing future physics/interaction conflicts and making the system more modular and reliable.

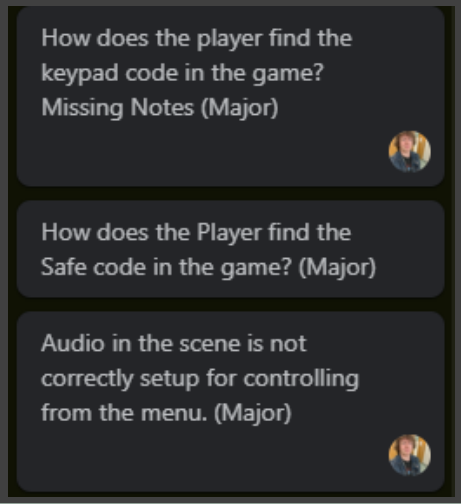
System Debugging: Implemented and utilised extensive debug logging to isolate issues between multiple keypad instances and trace the event flow from player interaction to puzzle completion, which was critical in identifying the root causes of the bugs.

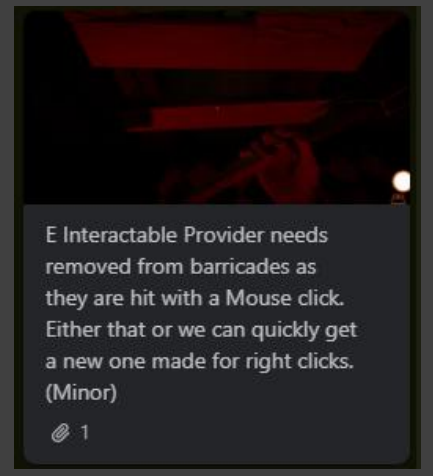
QA Testing – Bunker Scene (29/09/2025)

Today I completed a full QA test of the Bunker scene, ensuring it was fully playable from start to finish. I repeated the process twice to confirm consistent performance and reliability. Following the test, I created a dedicated Trello board entry titled *Bunker 1.0 (29/09/2025) Bugs and Changes Needed*, where I documented all issues discovered. Each issue was categorised as either Minor or Major to help the development team prioritise fixes, and every Trello card includes a detailed description of the problem, its exact location within the scene, supporting screenshots, and the assigned team member responsible for resolving it.









Updated Task Management Board (29/09/2025)

I have updated the Task Management Board on Trello by archiving all unnecessary task lists, ensuring that team members can easily navigate the board. Only the essential lists required for the final week of development remain, including the Debugging Lists for each scene. Additionally, I have renamed every list according to its corresponding scene, applying clear and organised naming conventions to improve overall board clarity and usability.

Final QA Test Of The Full Game (03/10/2025)

Today I did the final test run of the game before release, checking for bugs and ensuring that the game is fully playable from start to finish. Based on the results, I have updated the task management board on Trello after doing the final QA Tests to check for the final bugs and updates. I have filled out tasks for all four scenes, stating what needs to be fixed before release.